Izmir Institute of Technology

Electrical and Electronics Eng. Dep.

EE 471 Modern Software Development Practices and Technologies

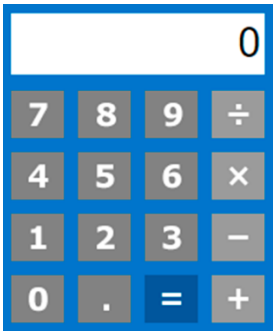
In-class exercise Week #4

Monday 19/10/2026

Preliminary study : Watch a Flask-based backend tutorial. Have a brief look on HTTP status code overview. Have Figma, GitHub and one AI coding tool ready (Claude Code, Codex, Cursor, etc.). Install one API testing tool: curl, Postman, or the Thunder Client extension in VSCode.

Section 1: CALCULATOR

Starting point: Use the merged main branch from Mini Project 1. Create a folder Week4, copy the calculator from Mini Project 1 into it, and work inside Week4 for all parts.



Part 1: A simple Figma design ready to vibe-code

1) Design the calculator above on Figma.
2) Use Auto Layout. Export a PNG and include a view-only link in your README.
3) Put the design/calculator.png and the Figma link in your README.
4) Push Week4 to your Git repo, committed as "section 1 – part 1".

Part 2: Vibe-code the frontend and a simple backend host via Flask

1) Use an AI coding tool of your choice to turn your design into a working web page (HTML/CSS/JS).
2) Move all calculations to a Flask server running on localhost. Flask must serve your page and expose POST /api/calculate using the JSON format given above. Do not use eval(). Return an error message for invalid input and division by zero.
3) Record your main prompts and any corrections you made in PROMPTS.md
   P.S: You are welcome to extend the calculator class you developed in Mini Project 1.
4) Push Week4 to your Git repo, committed as "section 1 – part 2".

Deliverables:

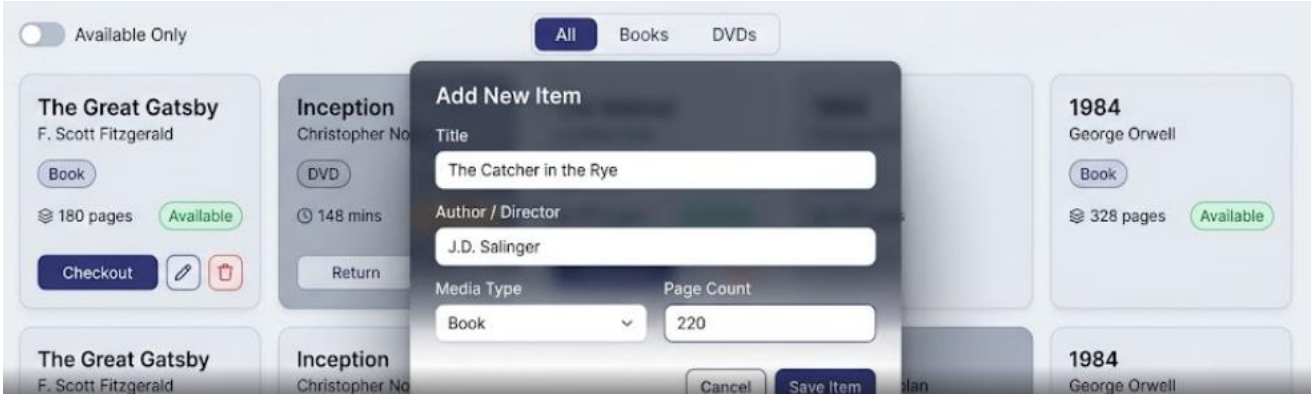
1) "video-section-1-1.avi": A screen recording video of you running the calculator on your localhost with a few different operations.
2) "video-section-1-2.avi": A screen recording video of you using API testing tools (e.g. Postman) to demonstrate these calculation operations running on your localhost.

Section Deliverable: "video-section-1.avi": A screen recording video of you explaining the function of each code block you use in your repository for Section 1.

Section 2: LIBRARY APP

Starting point: Use the merged main branch from Week 3. Copy custom_classes.py and main.py from Week 3 into your working directory and work inside Week4 for all parts.

Part 1: Figma design of the library app [BONUS]



Design the screens before coding, so you know what data each screen needs from the backend.

1. On Figma, design a single-page library web app with these elements:
   - Item list: one row or card per item showing title, author, type (Book/DVD), page count or duration, and an availability badge.
   - Actions per item: a Checkout/Return button, an edit icon, and a delete icon.
   - Add/Edit form: title, author, a type selector, and a field that switches between page count (Book) and duration in minutes (DVD).

- Filters: "Available only" toggle and a type filter (All / Book / DVD).
- States: an empty state (no items yet) and an error message area (e.g. "Cannot delete a checked-out item").

2. Use Auto Layout for the list and the form. Make the item row a component with two variants: *Available* and *Checked out*.
3. Put the design/library.png and the Figma link in your README.
4. Push Week4 to your Git repo, committed as "section 2 – part 1".

Part 2: Extend the classes with CRUD methods

Your Week 3 “custom_classes.py” can create and list items, but cannot find, edit or remove a single item. Complete the model in Python before building any web layer.

1. Inside custom_classes.py, create two exception classes: ItemNotFoundError and ItemStateError.
2. Give every item a unique integer id. LibraryCollection.add_item() assigns it (1, 2, 3, ...) and returns it. IDs are never reused, even after a delete.
3. Add a method to_dict() to MediaItem returning id, type, title, author and is_available. Override it in Book and DVD to add page_count or duration.
4. Change checkout() so it raises ItemStateError when the item is already out, instead of returning "Already out". Do the same in return_item() when the item is not checked out.
5. DVD.checkout() must no longer print() the reminder. Return it as part of the success message instead.
6. Add these methods to LibraryCollection:
    - get_item(item_id): returns the item, or raises ItemNotFoundError.
    - list_items(available=None, item_type=None): returns all items, optionally filtered.
    - update_item(item_id, **fields): updates only editable fields (title, author, page_count, duration). Changing id, type or is_available this way raises ValueError.
    - remove_item(item_id): removes the item. Raises ItemStateError if it is checked out.
7. In main.py, simulate one call of every method, including one that triggers each exception (catch them with try/except).
8. Push Week4 to your Git repo, committed as "section 2 – part 2".

Part 3: Flask REST API

Given the endpoints below (Every response is JSON. Error response {"error": "<message>"}.):

1. Expose your LibraryCollection through a REST API. The routes in app.py only translate HTTP to method calls; all rules stay in custom_classes.py.
2. Create app.py with one LibraryCollection instance, seeded with at least 3 items (mixed types). Create requirements.txt containing flask.
3. Push Week4 to your Git repo, committed as "section 2 – part 3".

Endpoint	Purpose	Success	Errors
POST /api/items	Create a Book or DVD	201 + the new item	400
GET /api/items	List items; supports ?available=true and ?type=book	200	400 bad filter
GET /api/items/<id>	Get one item	200	404
PUT /api/items/<id>	Replace all editable fields	200	400, 404
PATCH /api/items/<id>	Change some editable fields	200	400, 404
DELETE /api/items/<id>	Remove an item	204	404, 409 if checked out
POST /api/items/<id>/checkout	Check an item out	200	404, 409 already out
POST /api/items/<id>/return	Return an item	200	404, 409 not checked out

Example item:

{"id": 3, "type": "dvd", "title": "Arrival", "author": "Denis Villeneuve", "duration": 116, "is_available": false}

Deliverable: “video-section-2-3.avi”: A screen recording video of you testing each of the endpoints with your API testing tool (e.g. Postman)

Part 4: Vibe-code the frontend and connect it

1) Turn your Figma design into a working web page with an AI coding tool, and connect it to your API. Flask must serve the page: index.html, style.css, script.js. Open it at http://127.0.0.1:5000 (macOS users: if port 5000 is busy, use port=5001).
2) Show the API's error messages in your error area (e.g. try deleting a checked-out item and display the 409 message).
3) Review the generated code before you commit it. Write PROMPTS.md with your main prompts and at least two things the AI got wrong that you fixed (e.g. wrong status code handling, missing error display, logic duplicated in JavaScript instead of using the API).
4) Push Week4 to your Git repo, committed as "section 2 – part 4".

Section Deliverable: “video-section-2.avi”: A screen recording video of you explaining the function of each code block you use in your repository for Section 2.